

PART 4: SEQUENTIAL LOGIC

STATE REDUCTION AND MINIMIZATION

SAAD BAIG – DIGITAL LOGIC DESIGN

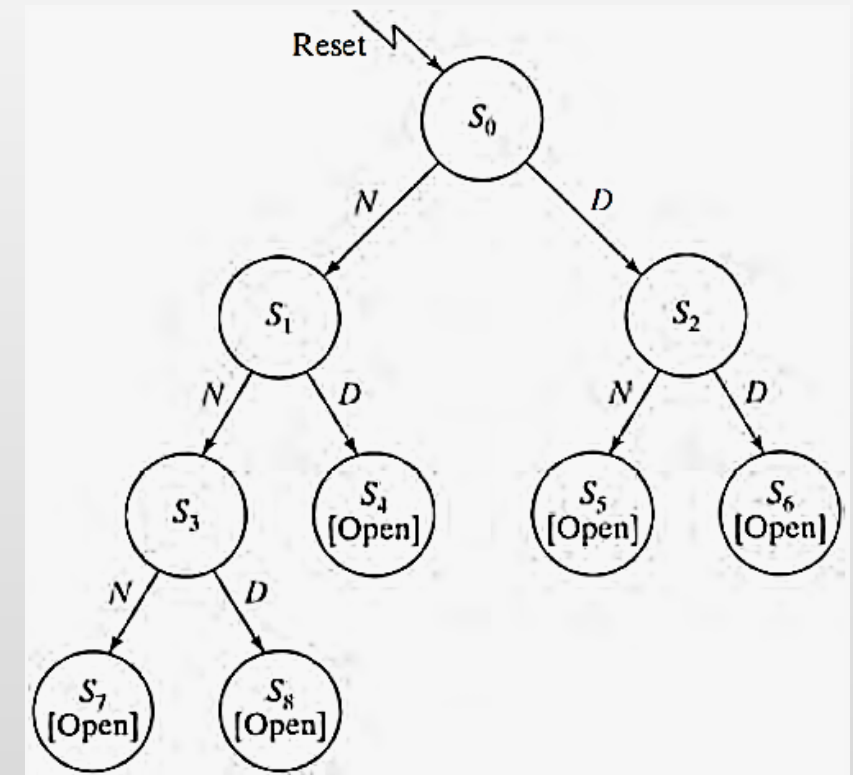


The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from <https://www.hasanbaig.com/resources>

© Hasan Baig and Junaid Ahmed Memon. All rights reserved. These files and accompanying materials are made available under the terms of “Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) License” and is available at <https://creativecommons.org/licenses/by-nc-nd/4.0/>

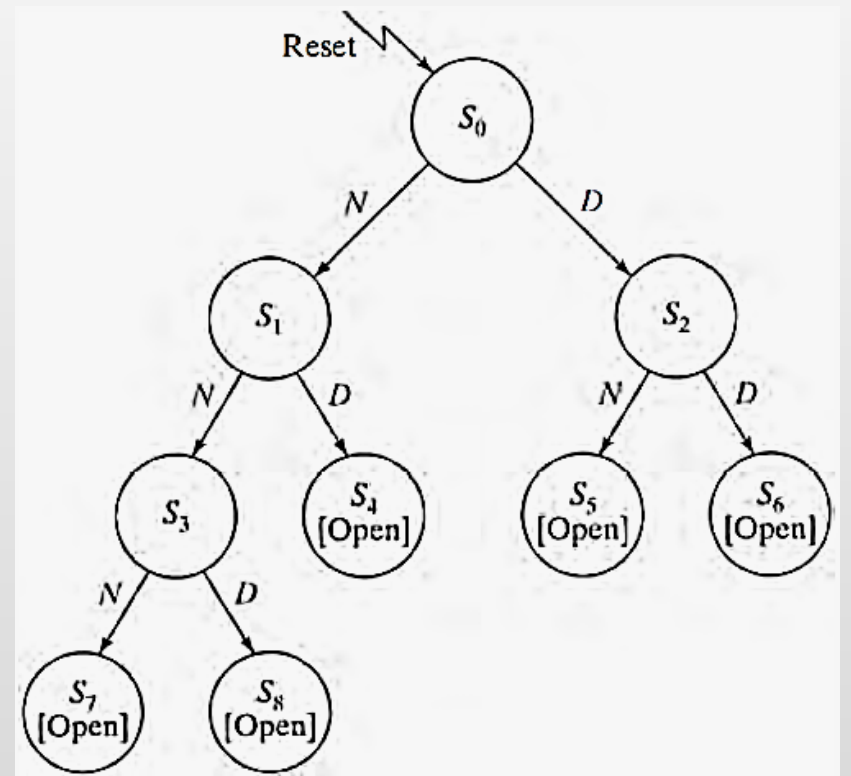
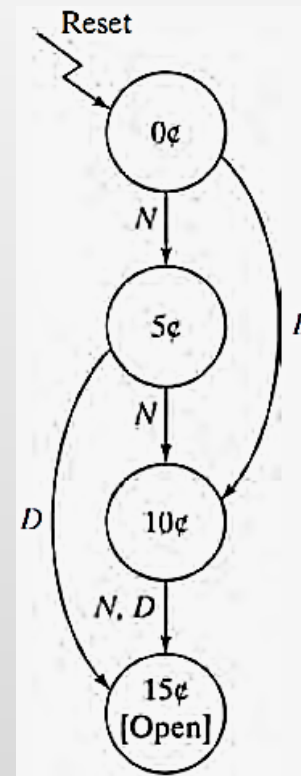
ABSTRACT REPRESENTATIONS

- Map the specification to a more suitable representation.
 - 3 nickels: N, N, N
 - 2 nickels 1 dime: N, N, D
 - 1 nickel 1 dime: N, D
 - 1 dime 1 nickel, D, N
 - 2 dimes: D, D
- Only transitions that cause a state change are included.
 - We assume that state remains same for no coin.



STATE MINIMIZATION

- S_4, S_5, S_6, S_7 and S_8 have identical behavior. They can be combined into a single state.
- We can also think of each state as representing the amount of money received so far.
 - It doesn't matter whether it took 1 dime or 2 nickels to reach a state with 10 cents.





STATE REDUCTION

- The reduction in the number of flip-flops in a sequential circuit is referred to as the **state-reduction** problem.
- State-reduction algorithms are concerned with procedures for reducing the number of states in a state table, while keeping the external input–output requirements unchanged.

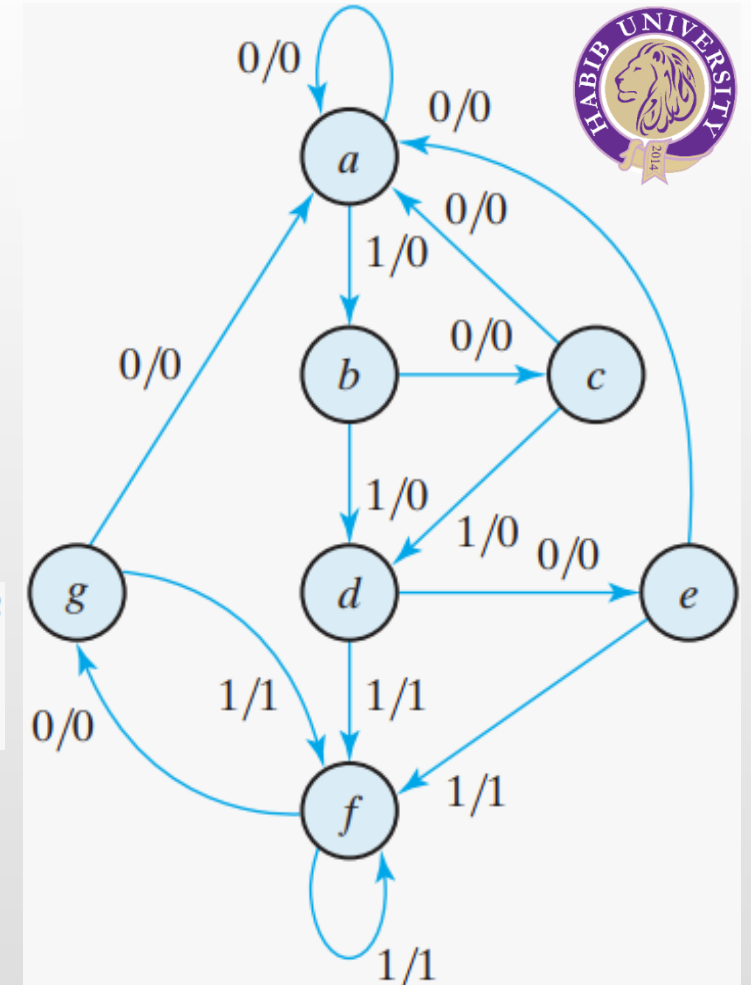
Two states are said to be equivalent if, for each member of the set of inputs, they give exactly the same output and send the circuit either to the same state or to an equivalent state.

- When two states are equivalent, one of them can be removed without altering the input–output relationships.

Input: 01010110100



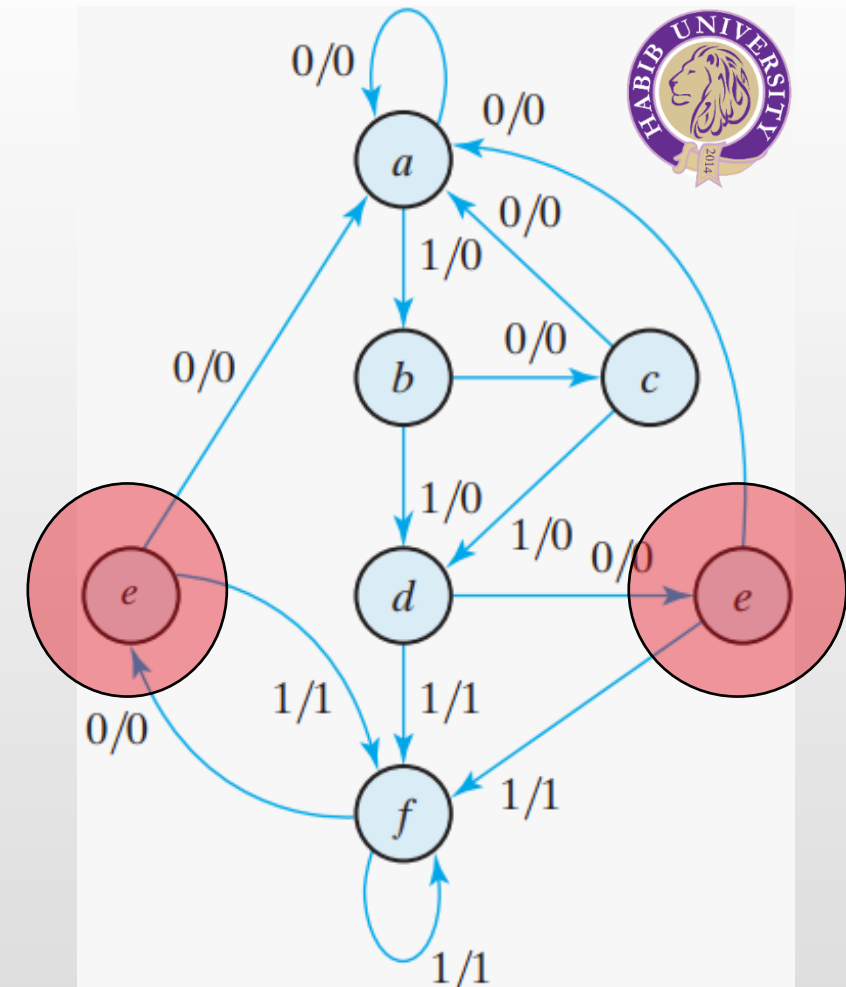
state	<i>a</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>f</i>	<i>g</i>	<i>f</i>	<i>g</i>	<i>a</i>
input	0	1	0	1	0	1	1	0	1	0	0	
output	0	0	0	0	0	1	1	0	1	0	0	



STATE REDUCTION EXAMPLE

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

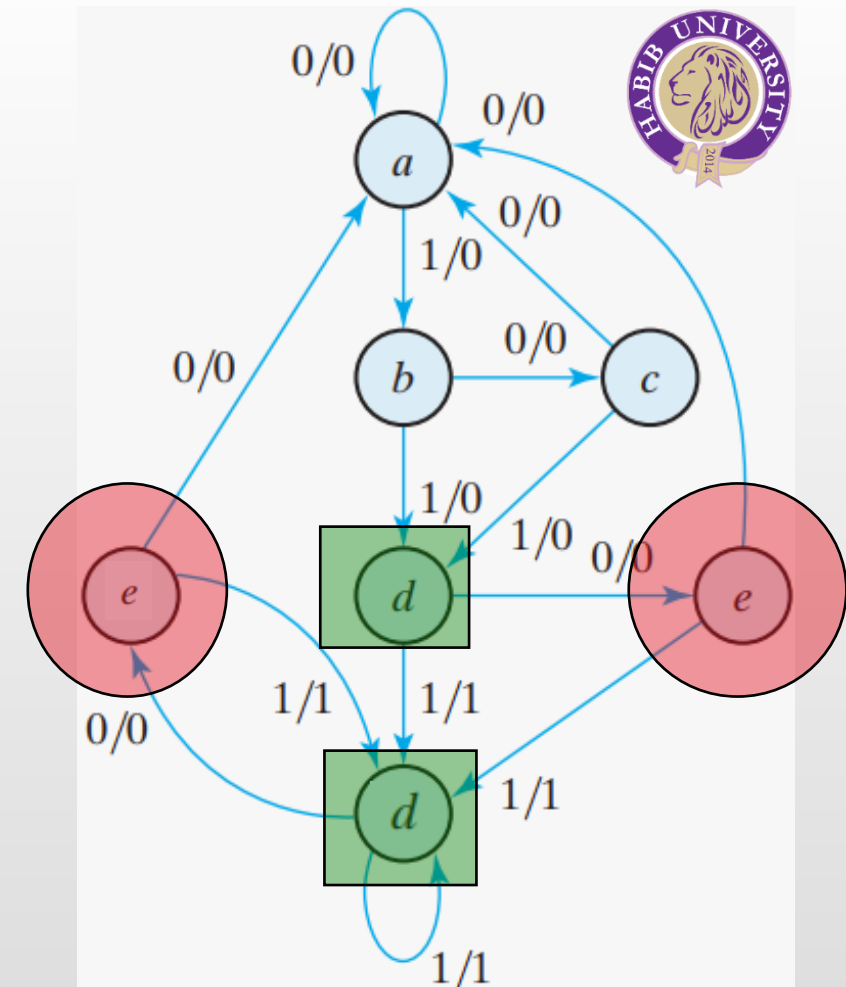
Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1



STATE REDUCTION EXAMPLE

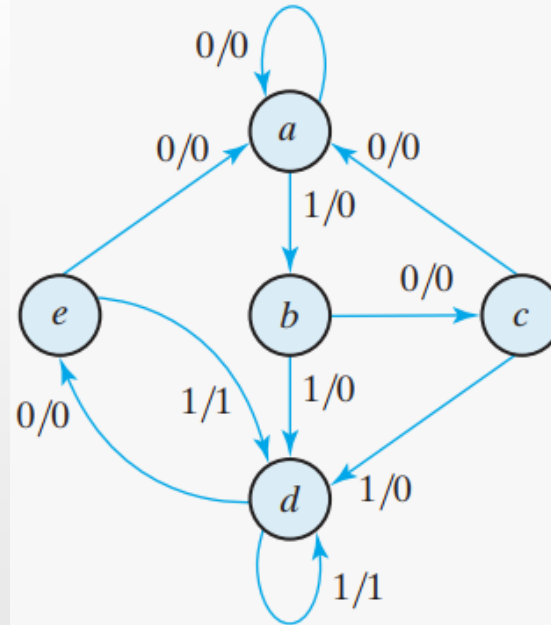
Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

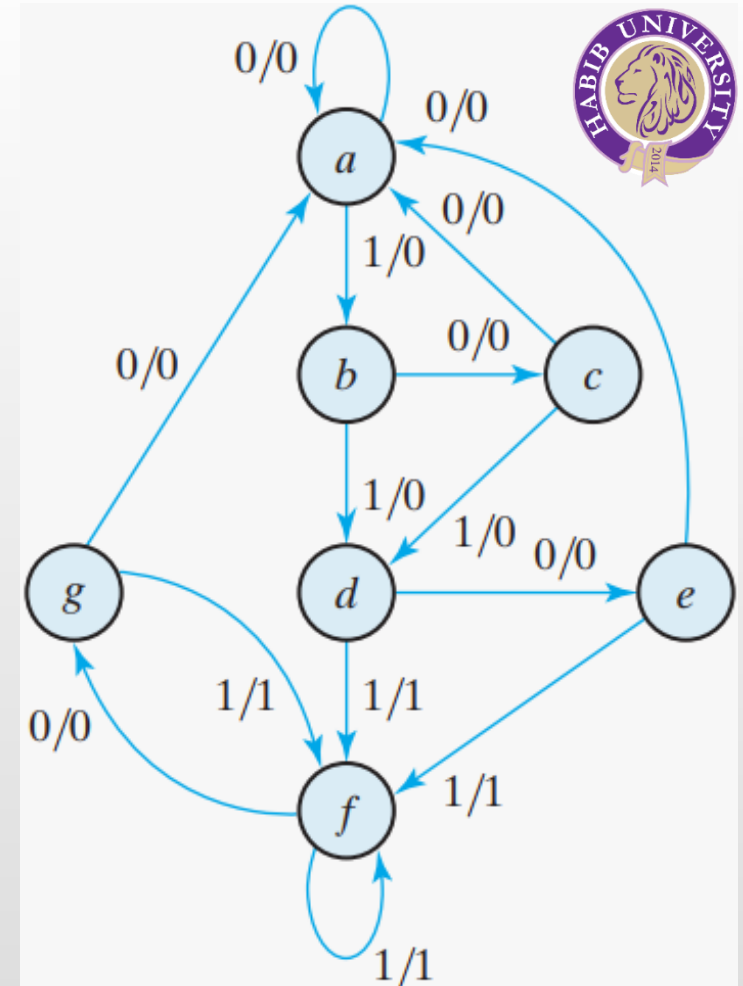


STATE REDUCTION EXAMPLE`

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1



state	a	a	b	c	d	e	d	d	e	d	e	a
input	0	1	0	1	0	1	1	0	1	0	0	
output	0	0	0	0	0	1	1	0	1	0	0	
state	a	a	b	c	d	e	f	f	g	f	g	a
input	0	1	0	1	0	1	1	0	1	0	0	
output	0	0	0	0	0	1	1	0	1	0	0	



STATE REDUCTION EXAMPLE



IMPLICATION CHART METHOD

- The implication chart method is a more systematic approach to finding the states that can be combined into a single reduced state.
- This method is more complex and better suited for machine implementation than hand use.

THREE-BIT SEQUENCE DETECTOR EXAMPLE

- Let's design a binary sequence detector that will output a 1 whenever the machine has observed the serial sequence 010 or 110 at the inputs.
- Initial State Table:

Input Sequence	Present State	Next State		Output	
		$X=0$	$X=1$	$X=0$	$X=1$
Reset	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_0	S_0	0	0
01	S_4	S_0	S_0	1	0
10	S_5	S_0	S_0	0	0
11	S_6	S_0	S_0	1	0



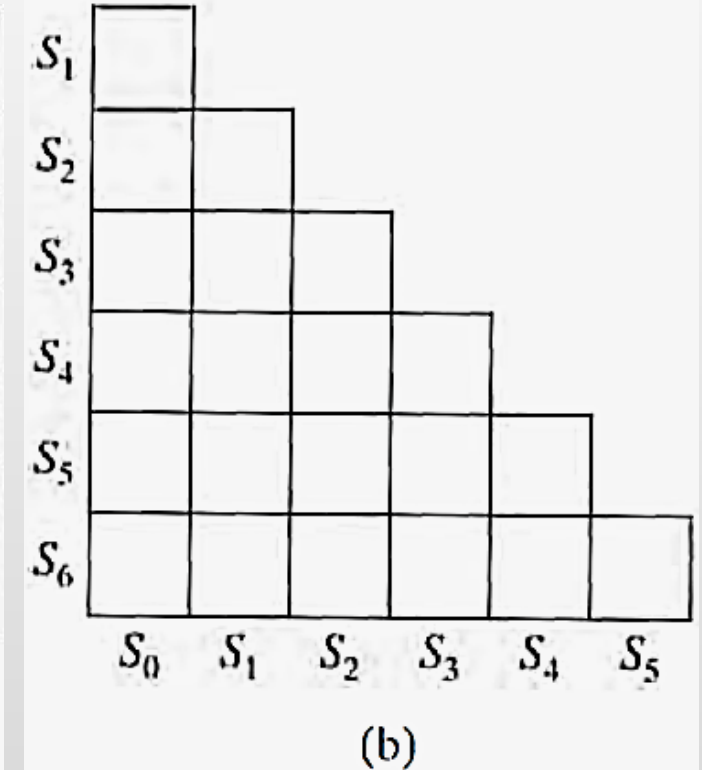
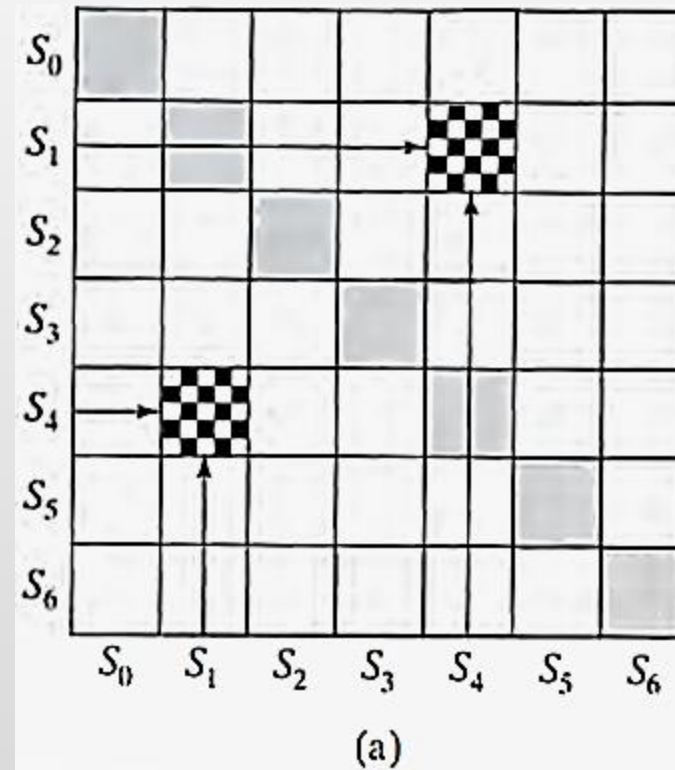
THREE-BIT SEQUENCE DETECTOR EXAMPLE

- Let's compare outputs.
- S_0, S_1, S_2, S_3 and S_5 have same outputs.
- States S_4 and S_6 also have same outputs.
- All other states have different outputs so they can never be equivalent.

Input Sequence	Present State	Next State		Output	
		$X=0$	$X=1$	$X=0$	$X=1$
Reset	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_0	S_0	0	0
01	S_4	S_0	S_0	1	0
10	S_5	S_0	S_0	0	0
11	S_6	S_0	S_0	1	0

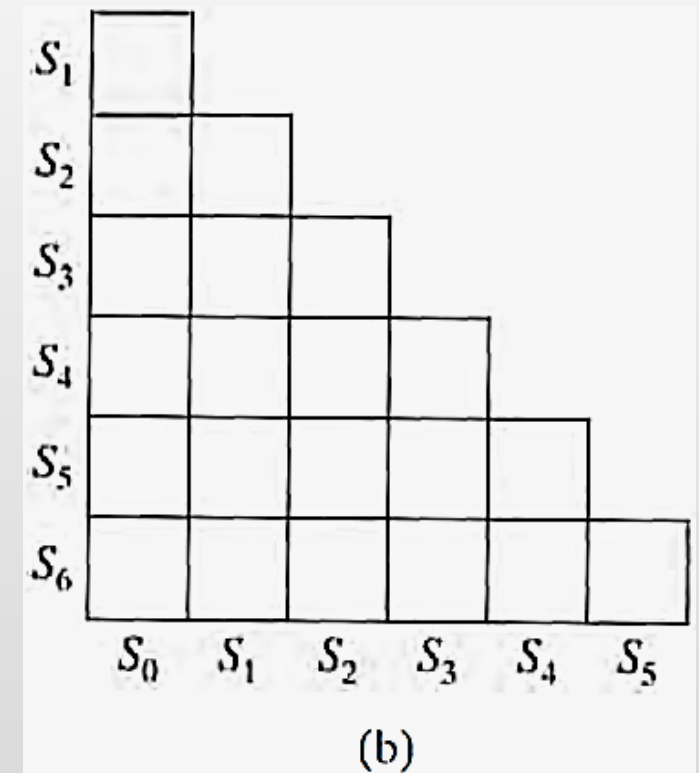
IMPLICATION CHART

- The method operates on a data structure that enumerates all possible combinations of states taken two at a time called an implication chart.
- a) This form of chart is more complicated (due to symmetry and repetition).
- b) We work with this structure.



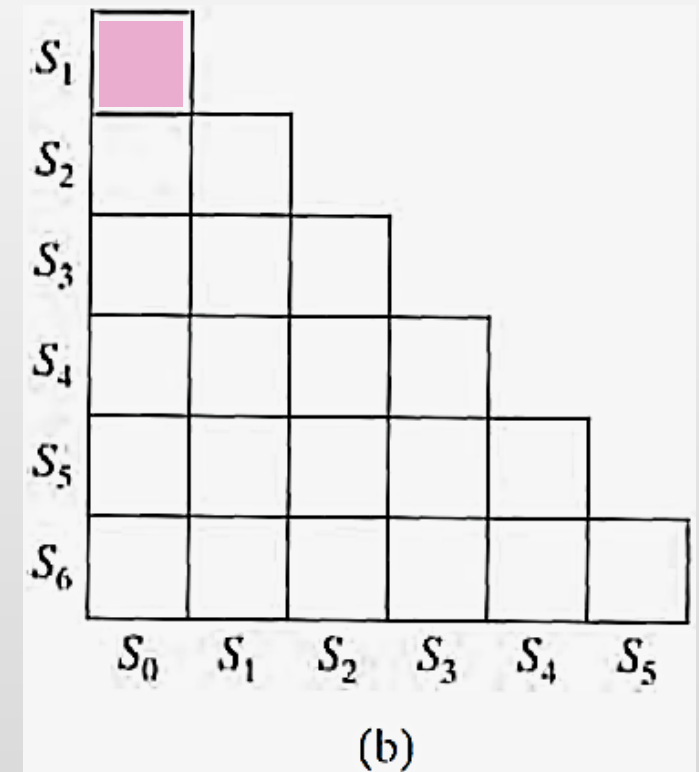
IMPLICATION CHART

- In general for n starting states, our implication data structure would require $(n^2 - n)/2$ cells.
- We will the chart as follows:
 - Let X_{ij} be the entry whose row is labelled by state S_i and column S_j .
 - If we were able to combine states S_i and S_j , it would imply that their next-state transitions for each input combination must also be equivalent.
 - If S_i and S_j have different outputs or next-state behavior, an X is placed. This means the states can never be equivalent.



INITIAL IMPLICATION CHART

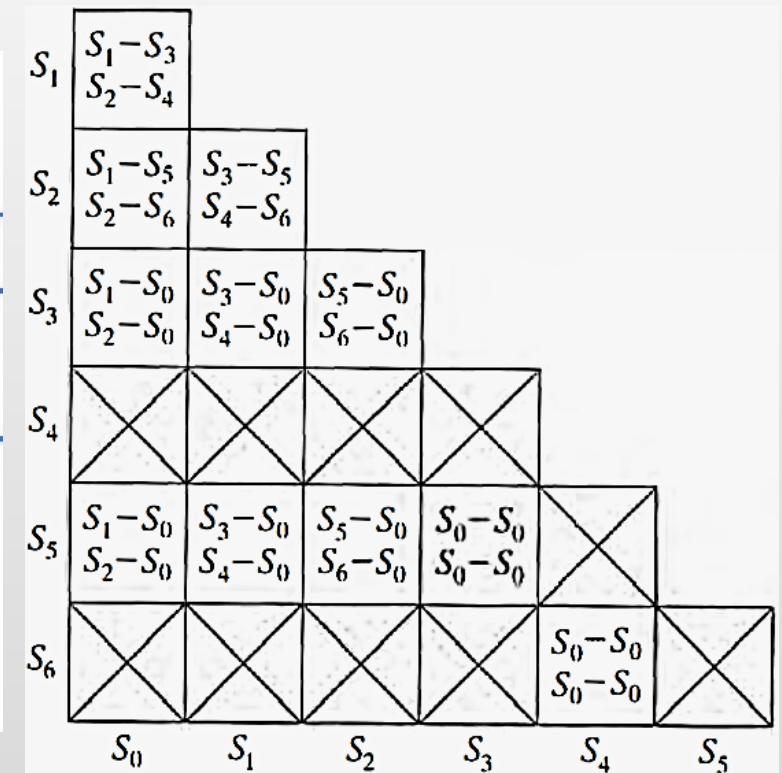
- To fill the box for S_1 (row) and S_0 (column), we will look at the next state transition.
 - S_0 goes to S_1 on 0 and to S_2 on 1.
 - S_1 goes to S_3 on 0 and to S_4 on 1.
 - We fill the chart with $S_1 - S_3$ to show transition for 0 and $S_2 - S_4$ for transition for 1.
 - This entry means S_0 and S_1 cannot be equivalent until S_1 is equivalent to S_3 AND S_2 is equivalent to S_4 .



-

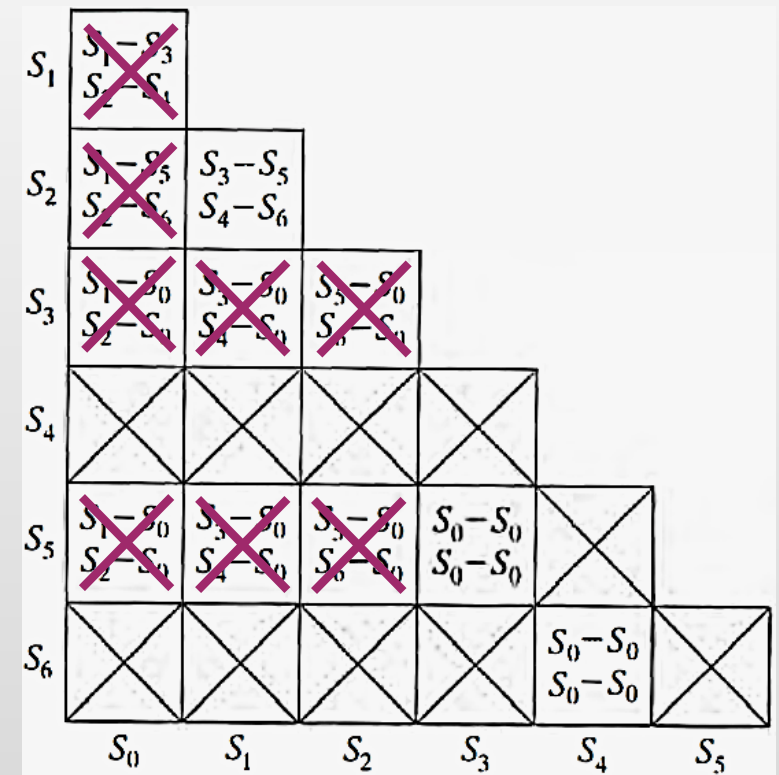
INITIAL IMPLICATION CHART

Input Sequence	Present State	Next State		Output	
		$X=0$	$X=1$	$X=0$	$X=1$
Reset	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_0	S_0	0	0
01	S_4	S_0	S_0	1	0
10	S_5	S_0	S_0	0	0
11	S_6	S_0	S_0	1	0



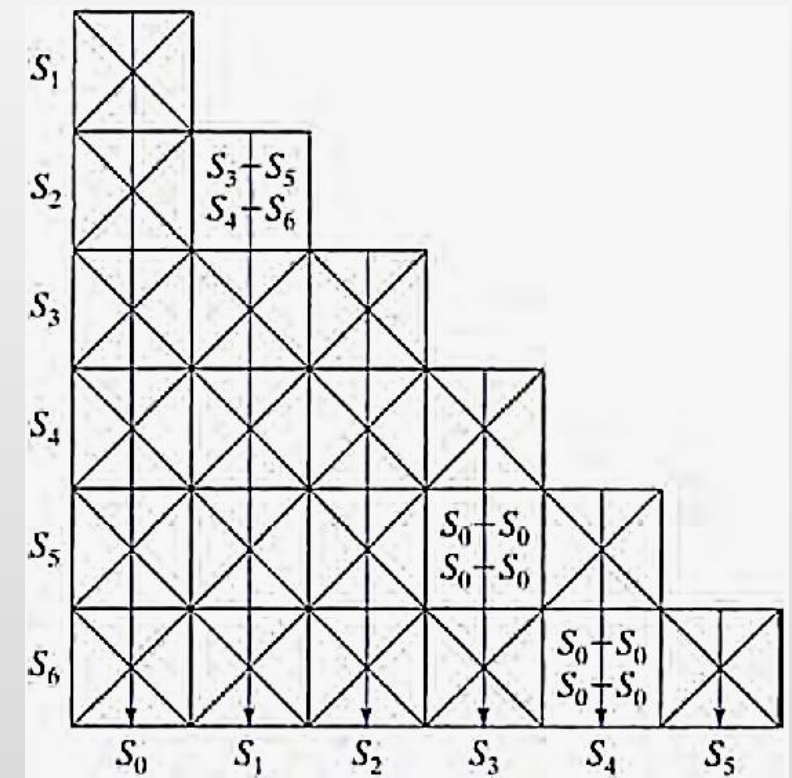
FIRST MARKING PASS

- At this point the chart already contains enough information to eliminate many impossible equivalent pairs.
- For example S_2 and S_4 can never be equivalent since they have different outputs. Hence S_0 and S_1 can never be equivalent.
- We start marking off the entries that will not be possible to combine.



SECOND MARKING PASS

- We now make a second pass through the chart to see if we can add any new markings.
- Since S_3 and S_5 will always be equivalent, along with S_4 and S_6 , entry S_2, S_1 will remain unmarked.
- Entries S_3, S_5 and S_4, S_6 are also obviously equivalent.
- So since there are no more new markings required, algorithm stops here. The unmarked entries represent equivalences between the row and column indices.
- S_1 is equivalent to S_2, S_3 to S_5 , and S_4 to S_6 .





REDUCED STATE DIAGRAM AND TRANSITION TABLE

- S_1 is equivalent to S_2 , S_3 to S_5 , and S_4 to S_6 .
- Let S_1 & $S_2 = S'_1$, S_3 & $S_5 = S'_3$, S_4 & $S_6 = S'_4$.

Input Sequence	Present State	Next State		Output	
		$X=0$	$X=1$	$X=0$	$X=1$
Reset	S_0	S_1	S_2	0	0
0	S_1	S_3	S_4	0	0
1	S_2	S_5	S_6	0	0
00	S_3	S_0	S_0	0	0
01	S_4	S_0	S_0	1	0
10	S_5	S_0	S_0	0	0
11	S_6	S_0	S_0	1	0

Input Sequence	Present State	Next State		Output	
		$X=0$	$X=1$	$X=0$	$X=1$
Reset	S_0	S'_1	S'_1	0	0
0 or 1	S'_1	S'_3	S'_4	0	0
00 or 10	S'_3	S_0	S_0	0	0
01 or 11	S'_4	S_0	S_0	1	0